Guess Market - Exercise 4 (bonus): the web client

Name: Dan Alushe

ID: 212472864

Email: danalushe15@gmail.com

GitHub: https://github.com/danal15/guess-market

The URL to open

http://localhost:3000

That is the whole of it. Open that address in a browser after running the batch file described below. There is no other page and no other port to remember.

How to run

- Start the server: put guess-market.war into tomcat\webapps and start Tomcat. The war is in this zip - it is the very same file submitted for exercise 3, so this runs against the server that was graded, not a rebuild of it. Nothing about the server changes for this exercise, and this client does not need it set up any differently.
- Start the web client: double click run-web.bat. It starts a small Node server and prints the address. Leave that window open while you are using the client; closing it stops the client.
- Open http://localhost:3000 in a browser. Chrome, Edge or Firefox are all fine.
- Log in with any name, the same as in the JavaFX client. There are no passwords. A name already in use is refused and you can pick another.
- To have two users at once, the easiest way is to run the desktop client as well: desktop-client\run.bat starts the JavaFX client from exercise 3, which is included here so both kinds of client can be seen working against the one server at the same time. Each copy of it you start is a separate user. A second browser user needs an incognito/private window or another browser - two ordinary tabs share one cookie and so are one user. That is how browsers work, and it is the one place where the web client cannot behave like two separate JavaFX processes.
- Nothing has to be downloaded or installed. The client uses only Node itself, with no npm packages at all, so it works on a machine with no internet.
- What has to be on the machine: Node (any version from 16 onwards - this was built and tested on Node 24) and a current browser. Chrome, Edge and Firefox from 2022 onwards are all fine; the client uses nothing newer than that.
- If Tomcat is not on localhost:8080, pass its address: run-web.bat http://localhost:9090
- If port 3000 is already taken on the machine, the window says so and tells you which port to ask for instead: set PORT=3001 and then run run-web.bat again.

Assumptions

Why there is a Node server at all: a browser will not let a page served from one address call a different one, and the session cookie would not be sent even if it did. The exercise says the client must not change how the server is run, so I could not add CORS headers to the war. Instead the small Node server does two jobs: it serves the page, and anything addressed to /guess-market it passes straight through to Tomcat and hands the answer back. The browser therefore only ever sees one address. Tomcat is untouched - it is started exactly as in exercise 3, and the JavaFX clients keep working against it at the same time.

Ports: the page is on 3000 and Tomcat on 8080. If 3000 is already taken the batch file says so and tells you how to pick another. If Tomcat is somewhere other than localhost:8080, set MARKET before running, for example: set MARKET=http://localhost:9090

No file upload: the exercise says the web user screen does not need it, so the web client does not have it. Events still arrive from files - somebody uploads them from the JavaFX client and they appear here on the next refresh. The web client can also create an event itself, which is the other way events come into being.

Chat: the exercise says bonuses from exercise 3 are not expected to work here. The chat is small and the server already has it, so I included it anyway, in the drawer at the bottom. It is not required and nothing else depends on it.

Reloading the page: pressing F5 does not log you out. The browser keeps the session cookie and the server never gives a name back once it is taken, so the page asks the server first whether it is already logged in. Without that, a reload would lock you out of your own account until the server restarted.

Nothing is saved: the market lives only while Tomcat runs, which was already true in exercise 3. Restarting the server clears the users, the events and the chat, and the page notices and returns to the login screen.

English only: all the text and all the input are English, left to right.

What is on the screen

- Login screen - a name, and what the server address is. It behaves the way the JavaFX one does: an empty name is refused without asking the server, a taken name shows what the server said, and the field is reselected so you can type over it.
- Events tab - every event in the system, from everybody's files, with the same three filters by method, status and commission, each with an "All". Clicking an event shows its details on the right: for LMSR the option prices, the event account, the trade history and the price chart; for an order book both books with their statistics, the bids and asks, the participants and the price chart. The column headings sort the table, and the sorting survives the refresh. This tab is view only, the same as in the JavaFX client.
- Account tab - your own screen and where the actions are. Your balance, "Load funds", every movement of money on its own line, a chart of the balance over time, the other users, and the list of events with what you are allowed to do in each one.
- The actions are the same ones the JavaFX client has: create an event, open it, buy shares in an LMSR event, place an order in an order book event, and close an event. A button you are not allowed to press right now is shown disabled with a tooltip saying why, rather than disappearing.

- Both trade forms ask the server what the trade would cost while you are typing, so the numbers are never a surprise after you confirm. The order form also says what is actually on the other side of the book and whether your order would trade at once or wait.
- Everything refreshes by itself once a second, so other people's trades, uploads and closings appear without pressing anything. If the server goes away the header says so quietly and keeps retrying, and it picks up again by itself when the server comes back.

What is in this folder

run-web.bat and web\ - the web client. This is what exercise 4 adds.

guess-market.war - the exercise 3 server, unchanged.

desktop-client\ - the exercise 3 JavaFX client (its jars and run.bat), unchanged. It is not part of exercise 4 and is here only so the two clients can be run side by side against the same server: an event uploaded or created in one appears in the other within a second.

What it is built from

Plain HTML, CSS and JavaScript, with no framework and no libraries. The exercise allows React or Angular or anything else, but every one of them would have meant files to download or a build step for the grader, and the screens here are tables and forms. The charts are drawn as SVG by hand for the same reason.

web\server.js - the Node server described above. It serves the files and passes the market calls through. It uses nothing but Node itself.

web\public\index.html - the three screens: login, events, account.

web\public\app.js - everything the page does. It talks to exactly the same endpoints the JavaFX client uses, and formats every number through the same rules the desktop Format class uses so the two clients never show the same figure differently.

web\public\style.css - the same colours and shapes as the desktop skin, so the two look like one product.

Nothing from the server is ever put into the page as markup. Event names, option names, descriptions, user names and chat lines all come from somebody's typing or from an uploaded file, so every one of them is added to the page as text. A name with a tag in it stays a name instead of becoming part of the page.

Working with the AI tool

Which tool, and did I try others: I used Claude, through Claude Code, which runs in a terminal and can read and edit the files in the project directly. I had already used it for exercises 2 and 3, so it knew the codebase. I also tried GitHub Copilot, which we get free as students. Copilot is good at finishing a line or a function while you type, but for this exercise I wanted something that could read the whole JavaFX client and the servlets and then write a matching client in one go, and the terminal tool was much better at that. The deciding factor was that it could run things: start the server, open a browser against it, click through the screens and read back what was actually on them.

Where the AI did not deliver, and where I had to step in: the first version looked right in the code and was broken in the browser. The login screen stayed on the screen underneath the market because a CSS class that sets display quietly beats the hidden attribute - the sort of thing you only find by looking at the running page.

It also wrote the first version as if the browser were just another Java client, and it took me pointing at the actual error to get the same-origin problem understood and the little proxy written. The AI is good at producing something plausible quickly and much weaker at knowing whether what it produced is true, so the useful division of labour was that it wrote and I insisted on evidence.

A bug it could not get on its own: in exercise 3 the client worked on my machine and not on my second computer. The AI kept looking at the login code. The real cause was that Java's HTTP client defaults to HTTP/2, which over plain http first asks Tomcat to upgrade the connection, and Tomcat answers a POST that asks this with 400, after which the call times out. I found that one myself by comparing what the two machines actually sent, and the fix was one line forcing HTTP/1.1. For this exercise the same pattern repeated on a smaller scale: it was quick to write the trade forms and slow to notice that asking for a quote of zero shares divides by zero on the server.

How much was the AI alone: most of the typing, and much less of the deciding. If I count lines of code, nearly all of it is the AI's. If I count the things that make it actually work - choosing the proxy over changing the server, insisting the pages be tested in a real browser rather than read, noticing that a reload locked you out of your own account - those came from going through it myself. I would put it at roughly three quarters written by prompt and a quarter where I had to understand what was happening and say what was wrong. I would not have been able to check its work at all without having written exercises 1 to 3 by hand first.

How long it took: about a day and a half with the tool, most of it testing and fixing rather than writing. On my own, with no front end experience, I think a week would have been optimistic - not because the screens are hard, but because I would have spent most of it learning what a fetch call and a CSS layout even look like.

Previous front end experience: none worth the name. I had written a little HTML at school and never any real JavaScript. It did not stop me finishing, but it did change what I was good for: I could not spot a wrong CSS rule by reading it, so I had to check everything by running it and looking.

Was it fun: more than the earlier exercises, and in a different way. The earlier ones were satisfying because I built them. This one was more like reviewing somebody else's work very closely, which is less fun than writing, but it is a lot faster and you get something finished that you could not have written alone. What I did not enjoy was how convincing the wrong answers are - it says everything in the same confident tone whether it is right or not, so you cannot relax.